

Bandit - Level 14

Goal

The password for the next level is retrieved by **submitting the password of the current level to port 30000 on localhost**.

Connection

```
ssh -i sshkey.private -p 2220 bandit14@bandit.labs.overthewire.org
```

Authenticated with the private key obtained in [Bandit - Level 13](#) — there is no password to type at login, which is precisely why the current password has to be read from disk.

Concepts

The subject shifts again: from authentication to **network services**.

localhost and the loopback interface

localhost is a hostname that always resolves to `127.0.0.1`, the **loopback** address: a virtual network interface pointing at the machine itself. Traffic sent there never reaches a cable or a switch — the kernel routes it straight back up the stack.

It's how a service and a client on the same host talk over TCP without involving the network at all. A database listening on `127.0.0.1:5432` is reachable by applications on that machine and by **nobody else**, which is a common and deliberate hardening choice.

Why localhost worked here but was blocked in [Bandit - Level 13](#): that restriction was specific to **SSH between levels**, imposed by OverTheWire to conserve server resources. It says nothing about TCP connections to other local services. Two unrelated things that happen to share the word "localhost".

Ports

A single IP address hosts many services at once. **Ports** are how the kernel tells them apart: a 16-bit number (0–65535) that, combined with the IP, identifies one endpoint of a connection.

Range	Name	Examples
0–1023	well-known	22 SSH, 25 SMTP, 53 DNS, 80 HTTP, 443 HTTPS
1024–49151	registered	3306 MySQL, 5432 PostgreSQL, 8080 HTTP-alt
49152–65535	dynamic / ephemeral	assigned to client connections

Port 30000 falls in the registered range and is arbitrary here — the level's service simply listens there.

On Linux, binding a port below 1024 requires root. That's why a web server drops privileges after binding 80, and why unprivileged services default to 8080.

nc — raw TCP

netcat opens a TCP connection to a host and port, then wires that connection to **stdin and stdout**:

```
nc localhost 30000
```

Everything typed goes over the network. Everything received is printed. No protocol, no framing, no encryption — hence its reputation as the network Swiss army knife.

Framed in familiar terms: **nc** is a pipe that crosses the network. The same stdin/stdout from [Linux - Piping and Redirection](#), with a socket in the middle instead of another process. Which is why it composes normally:

```
echo "password" | nc localhost 30000      # send and exit
cat /etc/bandit_pass/bandit14 | nc localhost 30000
nc -z localhost 30000-30010               # scan a range without sending data
nc -lvp 4444                             # LISTEN on a port instead of connecting
```

The connection is **interactive and stays open**: after connecting, the terminal appears to hang. It isn't frozen — it's waiting for input, and it transmits the moment Enter is pressed.

Attempts

```
cd ~/.ssh && cat authorized_keys
# → ssh-rsa AAAAB3NzaC1yc2E... rudy@localhost
# The PUBLIC half of the key pair from Level 13 – what the server uses to
# verify the client's signature. Interesting to see, but a dead end:
# it contains no password. Public keys are public by design.

ls -la ~
# → only .bashrc, .profile and .ssh – nothing to find in the home directory
```

Brief confusion over **which** password the level meant. The wording "*the password of the current level*" refers to bandit14's own password — the one Level 13 existed to obtain, now readable because the session is bandit14:

```
cat /etc/bandit_pass/bandit14
# → aaWecNkG4FhxJQxz07uiwzVP6bJiYS65
```

The same file that returned **Permission denied** one level earlier. Nothing about it changed; the identity reading it did.

```
nc localhost 30000
# (pressed Enter with no input)
# → Wrong! Please enter the correct current password.
```

Instructive rather than wasted: it proves the connection succeeded and the service is live. An empty line is still a line — **nc** transmits on Enter regardless of content, and the service evaluated it and rejected it.

Solution

```
cat /etc/bandit_pass/bandit14
# → aaWecNkG4FhxJQxz07uiwzVP6bJiYS65

nc localhost 30000
aaWecNkG4FhxJQxz07uiwzVP6bJiYS65
# → Correct!
#   pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
```

Non-interactive equivalent, avoiding the paste entirely:

```
cat /etc/bandit_pass/bandit14 | nc localhost 30000
```

The pipe feeds the file's contents into the socket. Cleaner, scriptable, and no risk of a mistyped character.

Points of Friction

- 1. Went looking for the answer in `~/.ssh` and `/home/bandit13`.** Reflex from the previous levels, where the target was always a file to find. Here nothing needed finding: the input was the identity already held. `authorized_keys` was a genuinely useful thing to look at — it shows how Level 13's authentication actually worked — but it was never going to contain a password.
- 2. Misread "the password of the current level".** Read as *the password needed to enter this level* rather than *the password belonging to this level's user*. They're the same string in every other level, which is why the distinction only surfaces here: Level 14 was entered with a **key**, so no password was ever typed.
- 3. Sent an empty line first.** Not really an error. The terminal looked frozen after connecting, Enter got pressed, and the service answered. That answer confirmed the connection was open and the service responsive — useful information, obtained by accident.

Key Takeaway

`nc` connects a TCP socket to stdin and stdout, which makes a remote service behave like any other stage in a pipeline. `cat file | nc host port` is the same composition already used a dozen times over, with the network standing in for a process.

The security point sits underneath: **the password crossed the wire in plaintext**. No encryption, no integrity check, no server authentication. On the loopback interface that's harmless, since the traffic never leaves the machine. Over a real network it would be readable by anyone in the path — the same exposure as HTTP Basic Auth described in [Linux - Encoding vs Encryption](#).

That is exactly the gap [Bandit - Level 15](#) closes: identical exchange, same kind of service, wrapped in TLS. Worth doing the two back to back and comparing, because the difference between `nc` and `openssl s_client` is the difference between HTTP and HTTPS.

Next

```
ssh -p 2220 bandit15@bandit.labs.overthewire.org
```

